



AWS ML Managed Services

Lecture Slides

These lecture slides are provided for personal and non-commercial use only

Please do not redistribute or upload these lecture slides elsewhere.

Good luck on your exam!

Amazon CodeGuru



Amazon CodeGuru is **machine-learning code analysis service**. CodeGuru performs code-reviews and will suggest changes to improve the quality of code. It can show visual code profiles (show the internals of your code) to pinpoint performance.

CodeGuru has three services:

- CodeGuru Security — detect, track and fix code security issues in code
 - Code Security Analytics Scan
 - Code Quality Analytics Scan
 - Secrets Detection Scan
- CodeGuru Profiler — find and fix inefficiencies in code
- CodeGuru Reviewer — associate a repo for continuous code change recommendations

GitHub Actions is used to automate continuous checks for GitHub repos

CodeGuru supports the following languages:

- Java
- JavaScript
- Python
- C#
- TypeScript
- Ruby
- Go
- IaC
 - CloudFormation
 - Terraform
 - AWS SDK (TypeScript or Python)

Amazon Comprehend



Amazon Comprehend is a **Natural Language Processor (NLP) service**.

Find relationships between text to produce insights.

Looks at data such as Customer emails, support tickets, social media and makes predictions

Amazon Comprehend can **analyze text and extract** the following:

- **Entities** — eg. Person, Organization, Location
 - **Key Phrases** — Text that appears important eg. You need to pay the amount of \$220.00 by August 8
 - **Language** — eg. confidence of the language being spoken eg. English
 - **PII (Personally identifiable information)** — eg. Eg. Andrew Brown, andrew@exampro.co
 - **Sentiment** — attitude towards the text — eg. 0.20 Negative
 - **Targeted sentiment** — specific words and their attitude eg. Awful 1.0 Negative
 - **Syntax** — identify parts of a language eg. Hello Proper Noun
 - **Custom Models** — upload your training data to analyze and extra custom text.
 - **Amazon Comprehend Flywheel** — automates the training of model versions for custom models.
-
- Amazon Comprehend is Serverless and you pay based on Size of request in units eg. 1 unit = 100 characters
 - Real-time analysis can be performed via a endpoint (or custom endpoint for custom models)
 - Analysis jobs allow for batch jobs

Amazon Comprehend

Entities — eg. Person, Organization, Location

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
-----------------	-------------	----------	-----	-----------	--------------------	--------

▼ **Analyzed text**

Hello world! This is Andrew Brown, showing off the amazing things that Amazon Comprehend can do!

Entity	Type	Confidence
Andrew Brown	Person	0.99+
Amazon	Organization	0.96
Comprehend	Commercial item	1.00

Amazon Comprehend

Key Phrases — Text the appears important eg. You need to pay the amount of \$220.00 by August 8

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
▼ Analyzed text						
Hello <u>world</u> ! This is <u>Andrew Brown</u> , showing off <u>the amazing things</u> that <u>Amazon Comprehend</u> can do!						

Key phrases	▼	Confidence
world		0.87
Andrew Brown		0.99+
the amazing things		0.99+
Amazon Comprehend		0.99+

Amazon Comprehend

Language — eg. confidence of the language being spoken eg. English

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
▼ Analyzed text						
Hello world! This is Andrew Brown, showing off the amazing things that Amazon Comprehend can do!						

Language

English, en

0.99 confidence

Amazon Comprehend

PII (Personally identifiable information) — eg. Eg. Andrew Brown, andrew@exampro.co

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
▼ Analyzed text						
Hello world! This is <u>Andrew Brown</u> , showing off the amazing things that Amazon Comprehend can do!						

Entity	Type	Confidence
Andrew Brown	Name	0.99+

Amazon Comprehend

Sentiment — attitude towards the text — eg. 0.20 Negative

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
▼ Analyzed text						
Hello world! This is Andrew Brown, showing off the amazing things that Amazon Comprehend can do!						

Sentiment

Neutral

0.01 confidence

Positive

0.98 confidence

Negative

0.00 confidence

Mixed

0.00 confidence

Amazon Comprehend

Targeted sentiment — specific words and their attitude eg. Awful 1.0 Negative

Entities

Key phrases

Language

PII

Sentiment

Targeted sentiment

Syntax

▼ Analyzed text

Positive

Neutral

Negative

Mixed

Hello world ! This is Andrew Brown , showing off the amazing things that Amazon Comprehend can do!

Entity ▼	Entity type ▼	Entity score ▼	Primary sentiment ▼	Positive score ▼	Negative score
<u>world</u>	LOCATION	0.83	NEUTRAL	0	0
<u>Andrew Brown</u>	PERSON	0.99+	POSITIVE	0.99+	0.00
<u>Amazon Comprehend</u>	ORGANIZATION	0.99+	NEUTRAL	0.00	0.00

Amazon Comprehend

Syntax — identify parts of a language eg. Hello Proper Noun

Entities	Key phrases	Language	PII	Sentiment	Targeted sentiment	Syntax
▼ Analyzed text						
Hello world! This is Andrew Brown, showing off the amazing things that Amazon Comprehend can do!						

Word	▼	Part of speech	▼	Confidence
Hello		Adjective		0.53
world		Noun		1.00
!		Punctuation		1.00
This		Pronoun		1.00
is		Verb		1.00
Andrew		Proper noun		1.00
Brown		Proper noun		1.00

Amazon Comprehend

The primary way you'll use Amazon Comprehend is via the AWS API.

Example of **analyzing text for language and sentiment** using Ruby SDK V3



```
require 'aws-sdk-comprehend'

client = Aws::Comprehend::Client.new
text = "Hello world! This is an example text for AWS Comprehend."

# Detect the dominant language
lang_resp = client.detect_dominant_language({text: text})
lang_code = lang_resp.languages.max_by(&:score).language_code
puts "Dominant Language: #{lang_code}"

# Detect sentiment
sentiment_resp = client.detect_sentiment({
  text: text,
  language_code: dominant_language_code
})

# Print the sentiment analysis results
puts "Sentiment: #{sentiment_resp.sentiment}"
puts "Positive: #{sentiment_resp.sentiment_score.positive}"
puts "Negative: #{sentiment_resp.sentiment_score.negative}"
puts "Neutral: #{sentiment_resp.sentiment_score.neutral}"
puts "Mixed: #{sentiment_resp.sentiment_score.mixed}"
```

Amazon Forecast



Amazon Forecast is a **time-series forecasting service**. Forecast business outcomes such as product demand, resource needs or financial performance.

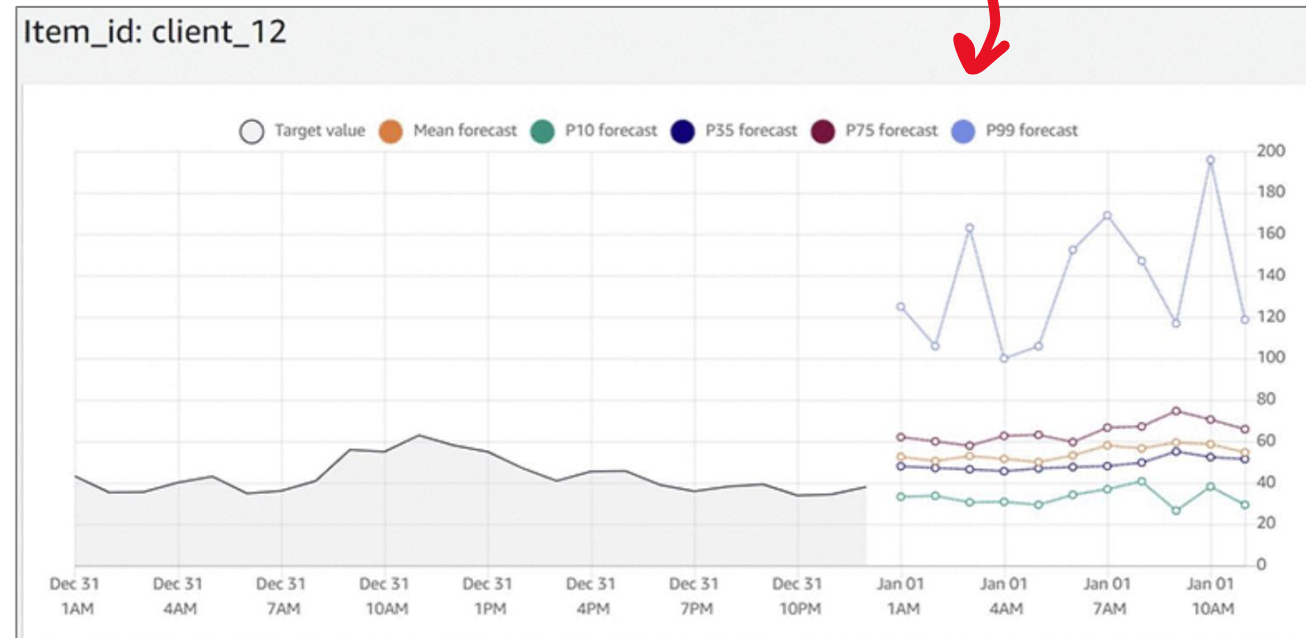
You need to upload your dataset to S3 with:

- Historical data
- Additional Metadata (optional)

Amazon Forecast Workflow:

- Create Data Set Group / Create Data Import Job
 - Define the schema
 - Register the task
- Create Predictor / Get Accurate Metrics
 - ELT Job evaluates the Model
 - Choose a predefined backtest
- Create Forecast
 - Deploy the Predictor
 - Retrained with full dataset
- Query Forecast / Export Forecast

Amazon Forecast will produce a **visual graph**



Amazon Fraud Detector



Amazon Fraud Detector is a **fully managed fraud detection a service**. identify potentially fraudulent online activities such as online payment fraud and the creation of fake accounts.

Amazon Fraud Detector comes with the following predefined models which you'll train your data against:

- **Online Fraud Insights** — optimized to detect fraud when little historical data is available about the entity being evaluated, for example, a new customer registering online for a new account.
- **Transaction Fraud Insights** — testing fraud use cases where the entity that is being evaluated might have a history of interactions that the model can analyze to improve prediction accuracy
- **Account Takeover Insights** — if an account was compromised by phishing or another type of attack.



Using the AWS SDK real-time fraud detections systems can be architected using AWS Step Functions, Amazon Kinesis, AWS Lambda and other AWS application integration services



You upload your dataset for data model training to an S3 bucket which is then reference by Fraud Detector

Amazon Fraud Detector

We **create our model**:

- Choose the Model Type eg. Online Fraud Insights
- Choose The Data Source eg. S3
- Define the Data Schema
 - Defined the label mapping
- Define the Model Variables to be used

After we review our model performance we set it **active** to deploy our model for real-time detection

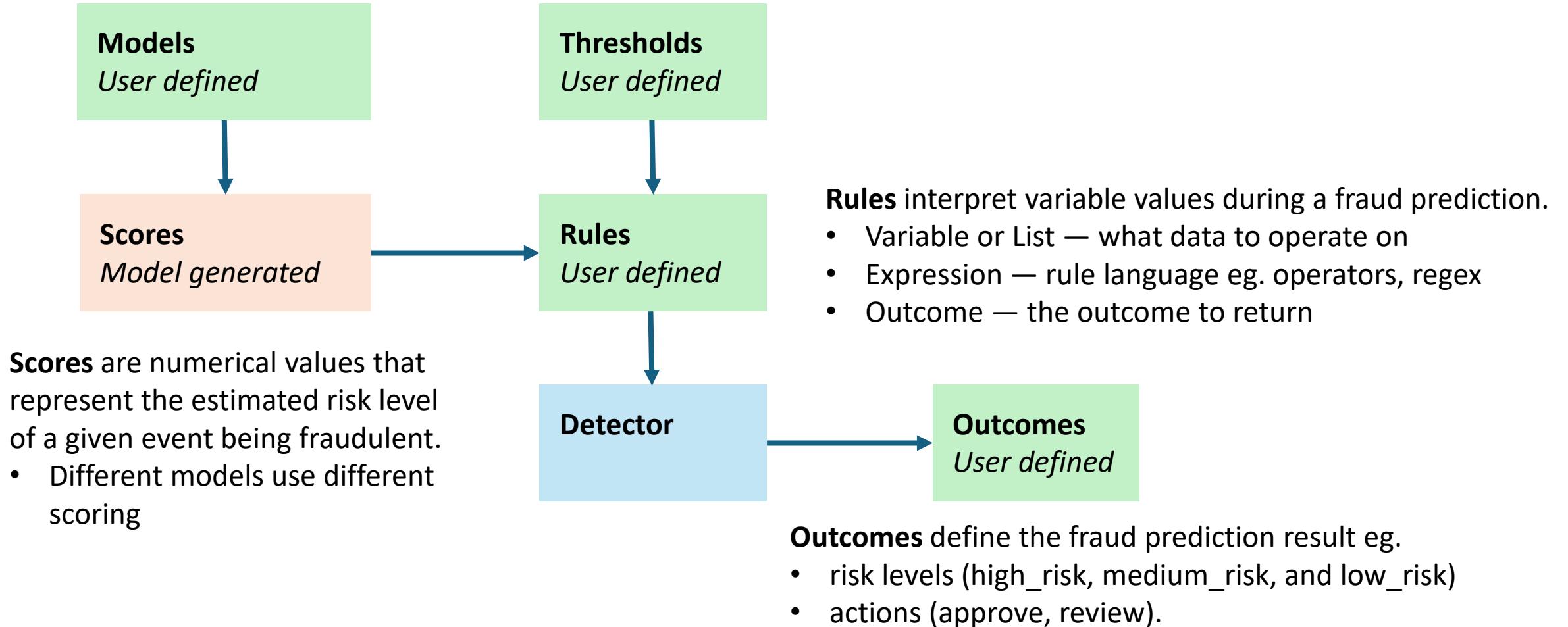
```
import boto3
fraudDetector = boto3.client('frauddetector')

fraudDetector.update_model_version_status (
    modelId = 'sample_fraud_detection_model',
    modelType = 'ONLINE_FRAUD_INSIGHTS',
    modelVersionNumber = '1.00',
    status = 'ACTIVE'
)
```

```
import boto3
fraudDetector = boto3.client('frauddetector')

fraudDetector.create_model_version (
    modelId = 'sample_fraud_detection_model',
    modelType = 'ONLINE_FRAUD_INSIGHTS',
    trainingDataSource = 'EXTERNAL_EVENTS',
    trainingDataSchema = {
        'modelVariables' : ['ip_address', 'email_address'],
        'labelSchema' : {
            'labelMapper' : {
                'FRAUD' : ['fraud'],
                'LEGIT' : ['legit']
            }
        },
        unlabeledEventsTreatment = 'AUTO'
    },
    externalEventsDetail = {
        'dataLocation' : 's3://bucket/file.csv',
        'dataAccessRoleArn' : 'role_arn'
    }
)
```

Amazon Fraud Detector



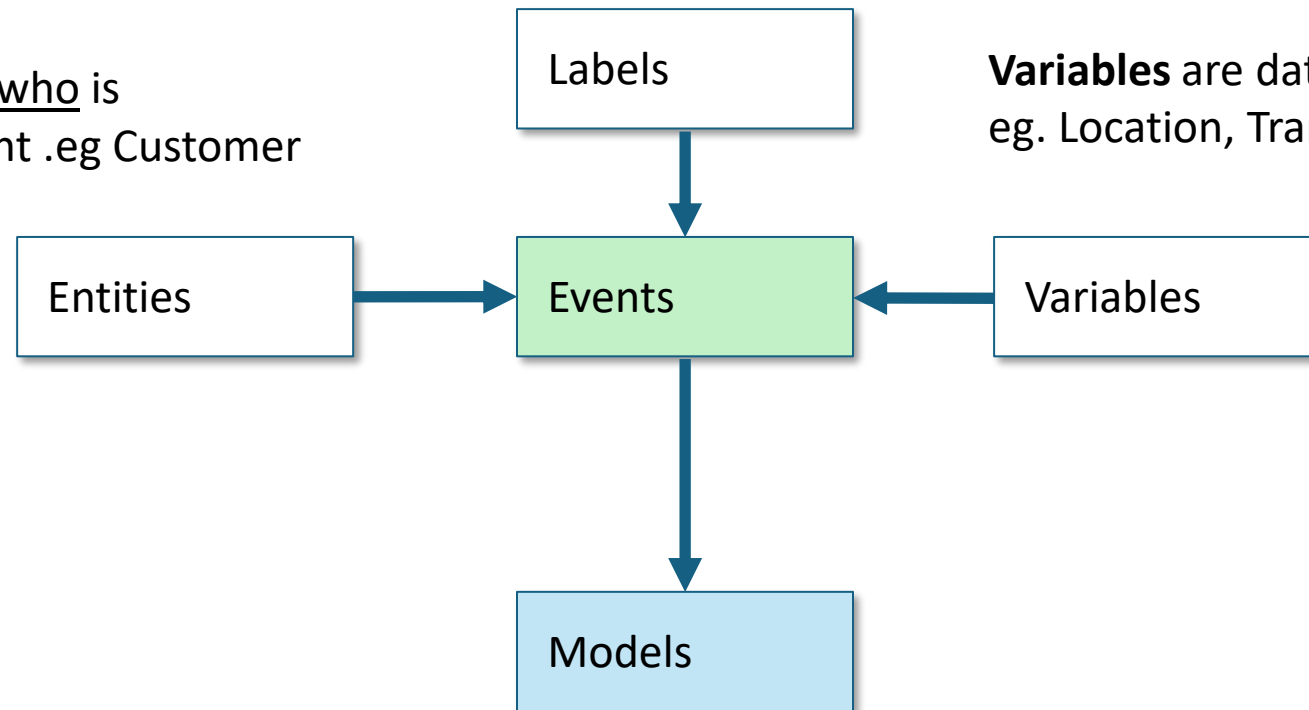
Amazon Fraud Detector

To defined an event type we need define **Labels, Entities and Variables**.

Labels classifies an event as fraudulent or legitimate.

Entities represents who is performing the event .eg Customer

Variables are data points used in your model.
eg. Location, Transaction Ammount.



Events are containing the data and rules that will be analyzed by the model

Amazon Kendra



Amazon Kendra is a **enterprise machine learning search engine service**. Uses natural language to suggest answers to question instead of just simple keyword matching



Instead of keyword-based search, Amazon Kendra uses **semantic and contextual understanding capabilities** to a search query. Its like interacting with a human.



Amazon Lex chatbot can be used as an interface to Amazon Kendra

Amazon Kendra has the following components:

- **Index** — a table that holds the index of your documents to make it searchable
- **Data Source** — Where your documents are stored eg. S3, Sharepoint, Box, Postgres, you need to define a schema.
 - **Data Source Template Schemas** — AWS provides around 40 schema templates for common AWS services or third-party cloud storage services
- **Document Addition API** — An API to add documents directly to an index

Amazon Kendra

Kendra has two versions which provides all features but with different limitations

Developer Edition

5 indexes with up to 5 data sources each

10,000 documents or 3 GB of extracted text

4,000 queries per day or 0.05 queries per second

Runs in 1 AZ

Enterprise Edition

5 indexes with up to 50 data sources each

8,000 queries per day or 0.1 queries per second

Runs in 3 AZs

**Developer Edition has free tier with 750 hours first 30 days*

Amazon Kendra

Create **Index**



```
aws kendra create-index \  
--name my-index \  
--description "My Index" \  
--role-arn arn:aws:iam::123456789012:role/KendraRoleForMyIndex
```

Create a **Data Source**



```
aws kendra create-data-source \  
--index-id index id \  
--name data source name \  
--role-arn arn:aws:iam::123456789012:role/KendraRoleFordDataSource \  
--type S3 \  
--configuration '{"S3Configuration":{"BucketName":"my-bucket"}}'  
# different data source would use a template to defined the schema  
# --type TEMPLATE \  
# --configuration '{"TemplateConfiguration":{"Template":{"JSON schema}}}'
```

You can defined your custom schema json for data source

Amazon Kendra

Sync the data source to the Index (update the index)

```
aws kendra start-data-source-sync-job \  
  --id <data_source_id> \  
  --index-id <index_id>
```



Query Kendra Index for results

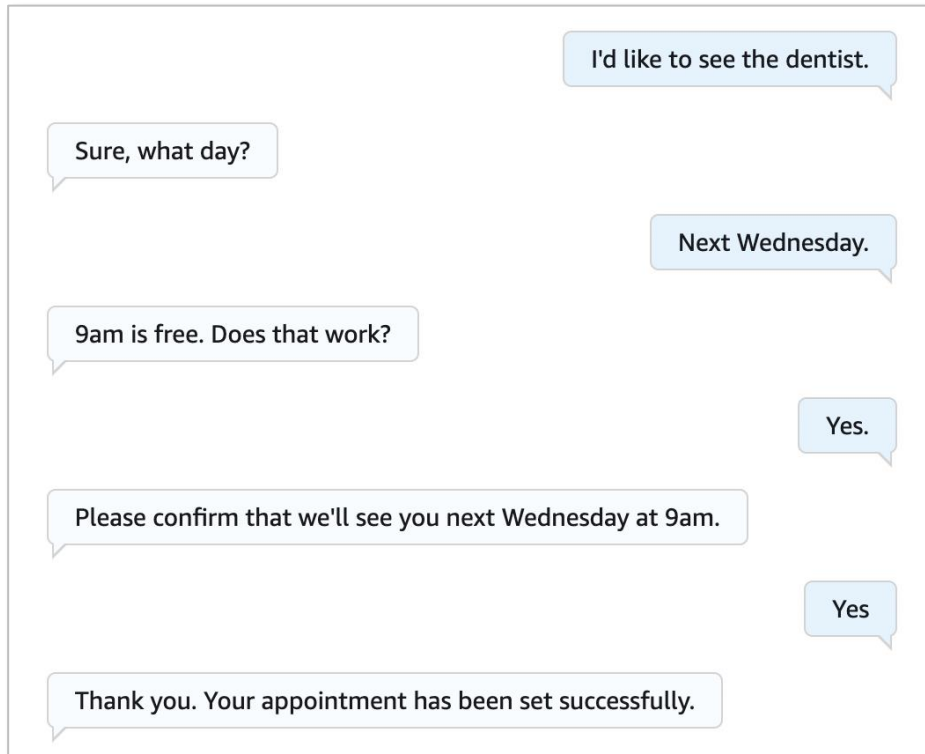
```
aws kendra query \  
  --index-id <index_id> \  
  --query-text "Do you have any affordable self sealing stem bolts for purchase?"
```



Amazon Lex



Amazon Lex (V2) is a **conversion interface service**. With Lex you can build conversational **voice and text chatbots**



Amazon Lex V2 provides

- Natural Language Understanding (NLU)
- Automatic Speech Recognition (ASR)
- AWS provide multiple bot templates for common industries as a starting points.
- Provide transcripts to create a new bot
- Use Gen AI to build a bot by describing what you want
- Choose a target language, can choose from multiple AWS provided voices.



Integrates with AWS Lambda to connect to various AWS services.



Amazon Lex Network Of Bots

A feature of lex that add multiple bots to a single network. A network can intelligently route the query to the appropriate bot. This provides a unified experience for customers and reduces duplication of intent configuration for multiple specialized bots.

Amazon Lex

Components of a Bot:

- **Bot** — performs automated tasks, the input to interact with your conversation model
 - **Version** — a number of version of the snapshot of your bot model
 - **Alias** — a label/tag to point to a specific version of a bot
 - **Language** — the target language(s) that the bot can converse in eg. English, Spanish
- **Intent** — represents an action that the user wants to perform.
 - **Sample Utterance** — text how a user might request their intent:
 - “Can I order a pizza”
 - “I want to order a pizza”
 - “Yo dawg, I want a slice a pie, can you dig it?”
 - **Slot** — inputs that an intent will require of the user (can be set to zero)
 - **Slot Type** — defines the data type of the slot
 - Can be a custom enumeration values eg. “Small”, “Medium”, “Large”
 - Can be a predefined data type eg. AMAZON.Number

Amazon Personalize



Amazon Personalize is a **real-time recommendations** service. Same technology used to make product recommendations to customers shopping on the Amazon platform

- Create a **data set group**
- Upload your **data set** to your data group (csv files)
 - You will provide multiple data sets
 - User Item Interaction Data
 - User Data
 - Item Data
 - You will need provide a json schema mapping for the CSV files
 - Reference the dataset location from an S3 object location
- **Solutions** and **Recipes** allows you to finetune your model
 - Solutions helps generate recommendations
 - Recipe is a predefined AWS algorithm
- **Event Trackers** — using the Ingestion SDK you can track user events and feed them to your solution
- **Filters** — allows you remove certain items from your recommendations based on rules
- Launch a **Campaign** to allow your applications to get recommendations from your solutions

Amazon Personalize

```
USER_ID,ITEM_ID,TIMESTAMP  
1,10,1577836800  
1,20,1577923200  
2,10,1578009600
```

User Item Interaction Data — core dataset that used to train a custom model

- At a minimum, this dataset must include three attributes
 - USER_ID: A unique identifier for a user.
 - ITEM_ID: A unique identifier for an item.
 - TIMESTAMP: The Unix timestamp of the interaction.

```
USER_ID,AGE,GENDER  
1,25,M  
2,34,F
```

User Data — contains metadata about the users, which can be used to improve recommendation quality

- The only required attribute in this dataset is the USER_ID, which must correspond to the USER_ID in the User-Item Interaction Data.

```
ITEM_ID,TITLE,CATEGORY  
10,"The Great Gatsby","Books"  
20,"Fender Stratocaster","Music Instruments"
```

Item Data — contains metadata about the items eg. categories, price, or brand.

- must include an ITEM_ID that matches the ITEM_ID in the User-Item Interaction Data.
- Must be called **CATEGORY_L1** (graphic to left is wrong)

Amazon Personalize

To get recommendations you'd use the AWS API

For example, using the **boto3** (AWS Python SDK)



```
import boto3

# Initialize the Personalize runtime client
personalize_runtime = boto3.client('personalize-runtime')

# Your campaign ARN (replace with your actual campaign ARN)
campaign_arn = 'your-campaign-arn'
# User ID to get recommendations for (replace with your actual user ID)
user_id = 'your-user-id'
# Optionally, you can specify an item ID if your scenario requires it
# item_id = 'your-item-id'
try:
    # Get recommendations
    response = personalize_runtime.get_recommendations(
        campaignArn=campaign_arn,
        userId=user_id,
        # Uncomment the next line if you need to specify an item ID
        # itemId=item_id
    )

    # Print the recommended items
    print("Recommended Items:")
    for item in response['itemList']:
        print(item['itemId'])
except Exception as e:
    print(e)
```

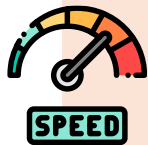
Amazon Polly



Amazon Polly is a **text-to-speech** service. Upload your text and an audio file spoken by synthesized voice is generated.

Engine Types:

- **Standard** (\$). — the standard model not as natural sounding as other engines, but most cost effective
- **Long Form** (\$\$) — sounds more natural when reading long forms of text
- **Neural** (\$\$\$). — Supports a Newscaster speaking style that is tailored to news narration use cases



There is variation between voices, depending on the text being spoke, no standard speed (words per minute) is available for Amazon Polly voices.

Lexicon — for specialized pronunciation of words.

A lexicon file (.xml, .pls) with upto of 40,000 characters and up to 100 pronunciation rules.

Speech Marks — metadata that describes the speech

- Where a word starts or ends
- Utilize Speech Synthesis Markup Language (SSML)
- Integrate with Visme (third party AI service to create marketing materials)

```
aws polly synthesize-speech \  
--engine neural \  
--output-format mp3 \  
--voice-id Joanna \  
--text "Hello, world!" \  
hello_world.mp3
```

Amazon Polly

Speech Synthesis Markup Language (SSML) is an XML-based markup language for speech synthesis applications.

```
<speak>  
  He was caught up in the game.<break time="1s"/> In the middle of the  
  10/3/2014 <sub alias="World Wide Web Consortium">W3C</sub> meeting,  
  he shouted, "Nice job!" quite loudly. When his boss stared at him, he repeated  
  <amazon:effect name="whispered">"Nice job,"</amazon:effect> in a  
  whisper.  
</speak>
```

Amazon Polly

SSML that Polly supports:

- `<speak>` — the root element
- `<break>` — pause
- `<emphasis>` — emphasis words eg. Strong, Moderate, Reduced
- `<lang>` — specify a different language
- `<mark>` — a custom tag for metadata
- `<p>` — pause between paragraphs
- `<phoneme>` — phonetic pronunciation for specific text
- `<prosody>` — controlling volume, speaking rate, and pitch
- `<s>` — Adding a pause between sentences
- `<say-as>` — Controlling how special types of words are spoken
- `<sub>` — Pronouncing acronyms and abbreviations
- `<w>` — Improving pronunciation by specifying parts of speech
- `<amazon:breath>` and `<amazon:auto-breaths>` — add breathing sound
- `<amazon:domain name="news">` Newscaster speaking style (only for Neural)
- `<amazon:effect name="drc">` — Adding dynamic range compression
- `<amazon:effect phonation="soft">` — speak softly
- `<amazon:effect vocal-tract-length>` — Controlling timbre
- `<amazon:effect name="whispered">` — Whispering

Amazon Rekognition



Amazon Rekognition is **image and video recognition service**. Analyze images and videos to detect and label objects, people, celebrities.

Amazon Rekognition has the following prebuilt models:

- Object Detection
- Face Detection
- Searching Faces in Connection
- People pathing
- Detecting personal protective equipment
- Recognizing celebrities
- Moderating content
- Detecting text
- Detecting video segments
- Detecting face liveness

For Images requirements:

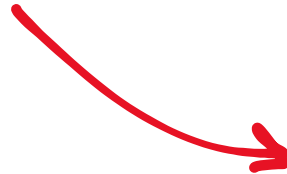
- Jpg or pngs
- Base64 encoding when passed via the HTTP API
 - AWS SDKs for Java, Javascript, Python and PHP will automatically base64 encoded.
- Can access images from S3 bucket



Amazon Rekognition Custom Labels can identify the objects, logos, and scenes in images that are specific to your business needs.

Amazon Rekognition – Object Detection

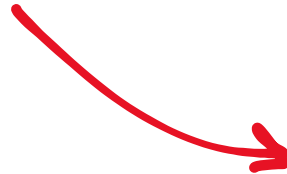
Detecting objects using the **detect_labels**



```
bucket = 'bucket' # the bucket name without s3://
photo = 'photo' # the name of file
client = Aws::Rekognition::Client.new
attrs = {
  image: {s3_object: {bucket: bucket, name: photo},
  }, max_labels: 10
}
response = client.detect_labels attrs
puts "Detected labels for: #{photo}"
response.labels.each do |label|
  puts "Label:      #{label.name}"
  puts "Confidence: #{label.confidence}"
  puts "Instances:"
  label['instances'].each do |instance|
    box = instance['bounding_box']
    puts "  Bounding box:"
    puts "    Top:      #{box.top}"
    puts "    Left:     #{box.left}"
    puts "    Width:    #{box.width}"
    puts "    Height:   #{box.height}"
    puts "  Confidence: #{instance.confidence}"
  end
  puts "Parents:"
  label.parents.each do |parent|
    puts "  #{parent.name}"
  end
  puts "-----"
  puts ""
end
```

Amazon Rekognition – Face Detection

Detecting faces using the **detect_faces**



```
bucket = 'bucket' # the bucketname without s3://
photo  = 'input.jpg'# the name of file
client  = Aws::Rekognition::Client.new
attrs = {
  image: {
    s3_object: {bucket: bucket,name: photo},
  },attributes: ['ALL']
}
response = client.detect_faces attrs
puts "Detected faces for: #{photo}"
response.face_details.each do |face_detail|
  low  = face_detail.age_range.low
  high = face_detail.age_range.high
  puts "The detected face is between: #{low} and #{high} years old"
  puts "All other attributes:"
  puts "  bounding_box.width:  #{face_detail.bounding_box.width}"
  puts "  bounding_box.height: #{face_detail.bounding_box.height}"
  puts "  bounding_box.left:    #{face_detail.bounding_box.left}"
  puts "  bounding_box.top:     #{face_detail.bounding_box.top}"
  puts "  mustache.value:       #{face_detail.mustache.value}"
  puts "  mustache.confidence:  #{face_detail.mustache.confidence}"
  puts "  eyes_open.value:      #{face_detail.eyes_open.value}"
  # ... many more options...
  puts "  confidence:          #{face_detail.confidence}"
  puts "-----"
  puts ""
end
```

Amazon Textract



Amazon Textract is a **OCR (extract text from scanned documents) service**. When you have paper forms, and you want to digitally extract the data.

Textract can:

- OCR documents
 - Can retain layout coordinates
 - Convert to a table
 - Detect forms
 - Query against the OCR data
 - Detect signatures
- **OCR expenses** eg. Receipts
- Analyze IDs (driver licenses or passports)
- Analyze Lending (mortgage documents)
- Custom Queries — train your own models with uploaded samples

WHOLE FOODS MARKET

Bryant Park BPK
1095 6th Ave
New York, NY 10036
917-728-5700

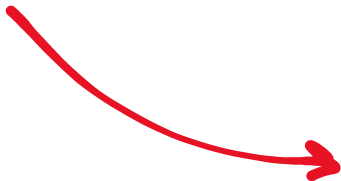
BROO BROWN ALE	\$10.99
BOTTLE DEPOSIT	\$0.30
DRSCL STRAWBERRIES	\$3.49
DVF OG LG EGGS	\$2.89
365 WHL MLK	\$4.09
NOOSA HONEY YOGHURT	\$2.29
365 OG ROMAINE BAG	\$2.69
365 SALTED CORN CHIPS	\$2.79
PDVG WHITE BAGIT	\$3.50
365 PNBTR BALLS OG	\$3.99
365 JMBO PAPER TOWELS	\$1.69
365 CHUNKY SALSA	\$2.69
LACRX GRAPEFRUIT	\$5.99
BOTTLE DEPOSIT	\$0.60
PNLND GRND BEEF	\$5.99
Subtotal:	\$53.98
Net Sales:	\$53.98
Tax	\$1.66
Total:	\$55.64
Sold Items:	13
Paid:	
VISA	\$55.64

04/02/2019 09:16:42

(CITY) Normalized Field: CITY New York,	(STREET) Normalized Field: STREET 1095 6th Ave
(ADDRESS) Normalized Field: ADDRESS Bryant Park BPK 1095 6th Ave New York, NY 10036	Subtotal: Normalized Field: SUBTOTAL \$53.98

Amazon Textract

Examples of **analyzing a document**
using the AWS Ruby SDK V3



```
textract_client = Aws::Textract::Client.new
bucket_name = 'your-bucket-name'
document_name = 'your-document-name'

begin
  response = textract_client.analyze_document(
    document: {
      s3_object: {
        bucket: bucket_name,
        name: document_name
      }
    },
    # Specify the features you want to analyze
    feature_types: ['TABLES', 'FORMS']
  )
  puts response.to_h
rescue Aws::Textract::Errors::ServiceError => e
  puts "Error calling Amazon Textract:"
  puts e.message
end
```

Amazon Translate



Amazon Translate **neural machine learning text translation service**. Uses deep learning models to deliver more accurate and natural sounding translations.

Processing modes:

- Real-time translation
 - Async batch processing
-
- Give it text
 - Choose a source language
 - Choose a target language

```
client = Aws::Translate::Client.new

text_to_translate = "Hello, how are you?"
begin
  response = client.translate_text({
    text: text_to_translate,
    source_language_code: 'en',
    target_language_code: 'es'
  })
  puts "Translated Text: #{response.translated_text}"
rescue Aws::Translate::Errors::ServiceError => e
  puts "Error translating text: #{e.message}"
end
```